Esper & EPL

resources

- espertech
- EPL documentation
- online environment to try EPL

let's get dirty

running example

> Count the number of fires detected using a set of smoke and temperature sensors in the last 10 minutes A fire is detected if at the same sensor a smoke event is followed by a temperature>50 events within 2 minutes

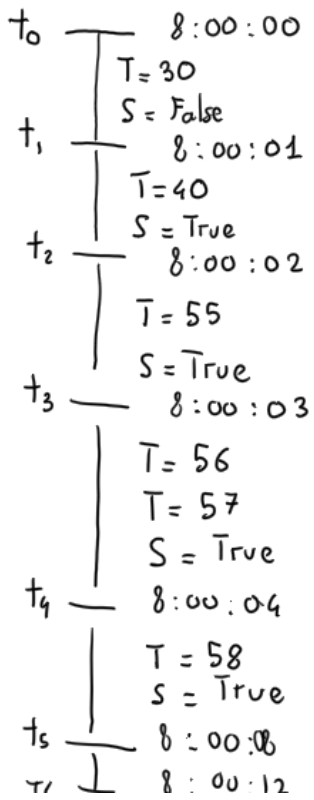
schema of the event types for the running example

```
create schema TemperatureSensorEvent (
sensor string,
temperature double
);

create schema SmokeSensorEvent (
sensor string,
smoke boolean
);

create schema FireEvent (
sensor string,
smoke boolean,
temperature double
);
```

data stream for the running example

```
TemperatureSensorEvent={sensor='S1', temperature=30}

SmokeSensorEvent={sensor='S1', smoke=false}

t=t.plus(1 seconds)

TemperatureSensorEvent={sensor='S1', temperature=40}

SmokeSensorEvent={sensor='S1', smoke=true}

t=t.plus(1 seconds)

TemperatureSensorEvent={sensor='S1', temperature=55}

SmokeSensorEvent={sensor='S1', smoke=true}

t=t.plus(1 seconds)

TemperatureSensorEvent={sensor='S1', temperature=56}
TemperatureSensorEvent={sensor='S1', temperature=57}

SmokeSensorEvent={sensor='S1', smoke=true}

t=t.plus(1 seconds)
TemperatureSensorEvent={sensor='S1', temperature=58}

SmokeSensorEvent={sensor='S1', smoke=true}

t=t.plus(4 seconds)
t=t.plus(4 seconds)
```

visually



let's explore the EPL syntax by example

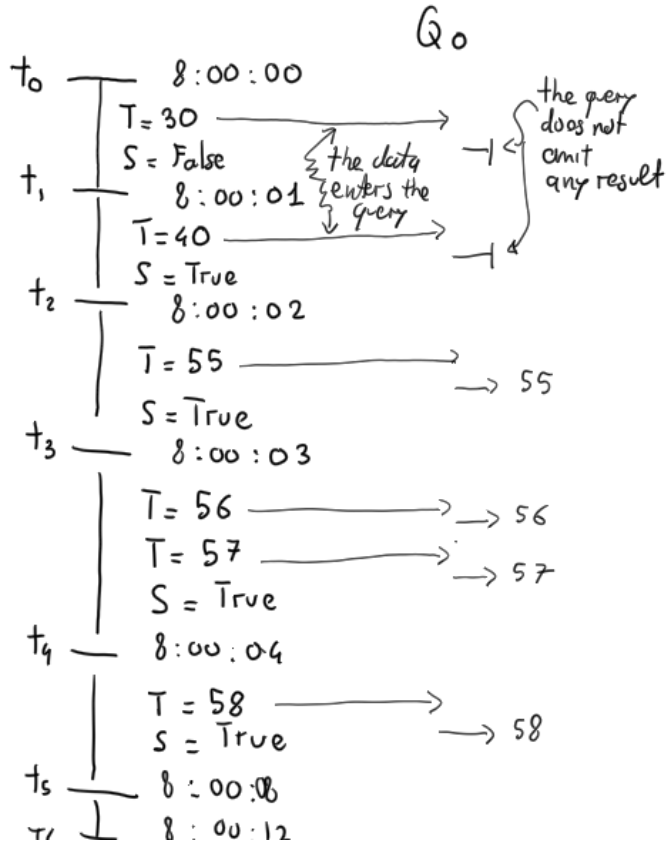
Q0

the temperature events whose temperature is greater than 50 C°

the SQL style

```
@Name('Q0')
select *
from TemperatureSensorEvent
where temperature > 50;
```

visually

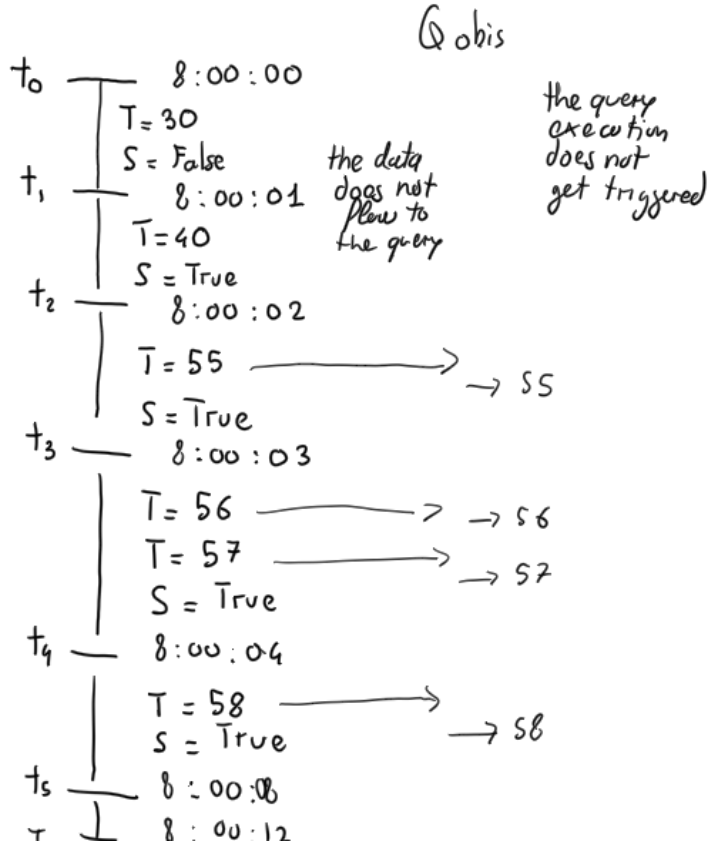


Execution mode: Data points enter the query that filters them

The Event-Based System Style

```
@Name('Q0bis')
select *
from TemperatureSensorEvent(temperature > 50) ;
```

visually



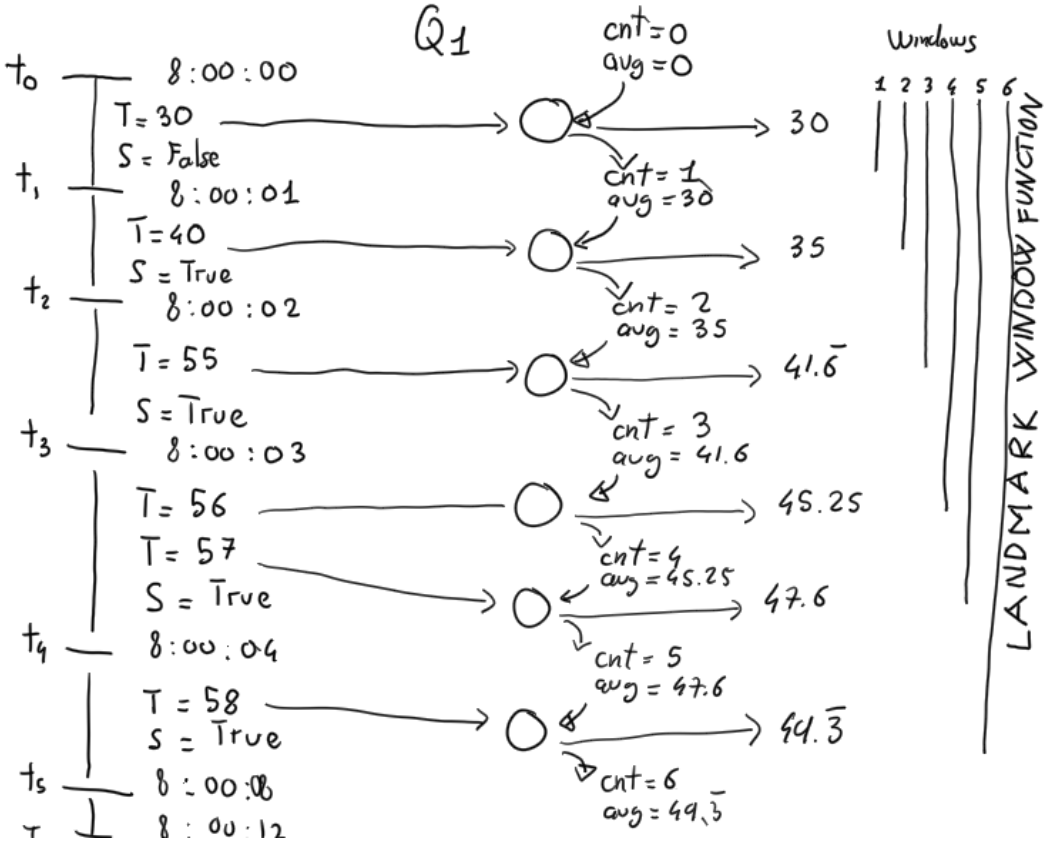
Execution mode: data points are filtered before flowing into the query and the query execution does not get triggered

Q1

the average of all the temperature observations for each sensor up to the last event received

```
@Name('Q1')
select sensor, avg(temperature)
from TemperatureSensorEvent
group by sensor;
```

visually



Execution mode

- From a logical perspective, data points cumulate in the landmark window, and the query emits a new avg for every data point
- From a physical perspective, the query is evaluated incrementally and maintains a state that captures the number of events (cnt) seen so far and the last avg. The new average depends on the state and the latest data point entering the query. Old data can be forgotten.

Not all the algorithms can be converted to a streaming one, e.g., there is not **an exact** way to compute the standard deviation in a streaming algorithm. However, there is an approximate streaming algorithm for the standard deviation that **assumes** the stationarity of the observed process.

NOTE: even if you cannot read it in the syntax, a *landmark window* opens when the stream starts (or when you register the query in esper) and keeps widening. As a side note, the *logical unbounded table* assumed by spark is a landmark window, so the two words are synonyms.

Q3

The events received in the last 2 seconds every 2 seconds (a.k.a. **logical tumbling window**)

```
@Name('Q3')
select *
from TemperatureSensorEvent.win:time_batch(2 seconds);
```

Q4

The last 2 events received (a.k.a. **physical tumbling window**)

```
@Name('Q3')
select *
from TemperatureSensorEvent.win:length_batch(2);
```

Q5

The events received in the last 4 seconds for every event (a.k.a. **logical sliding window**)

```
@Name('Q5')
select *
from TemperatureSensorEvent.win:time(4 seconds);
```

Note: it appears to emit only the event it receives when it receives it ...

Q6

The last 4 events received for every event (a.k.a. **physical sliding window**)

```
@Name('Q6')
select *
from TemperatureSensorEvent.win:length(2 seconds);
```

Note: it appears to emit only the event it receives when it receives it ... and it appears to generate exactly the same results of Q5 ...

Q7

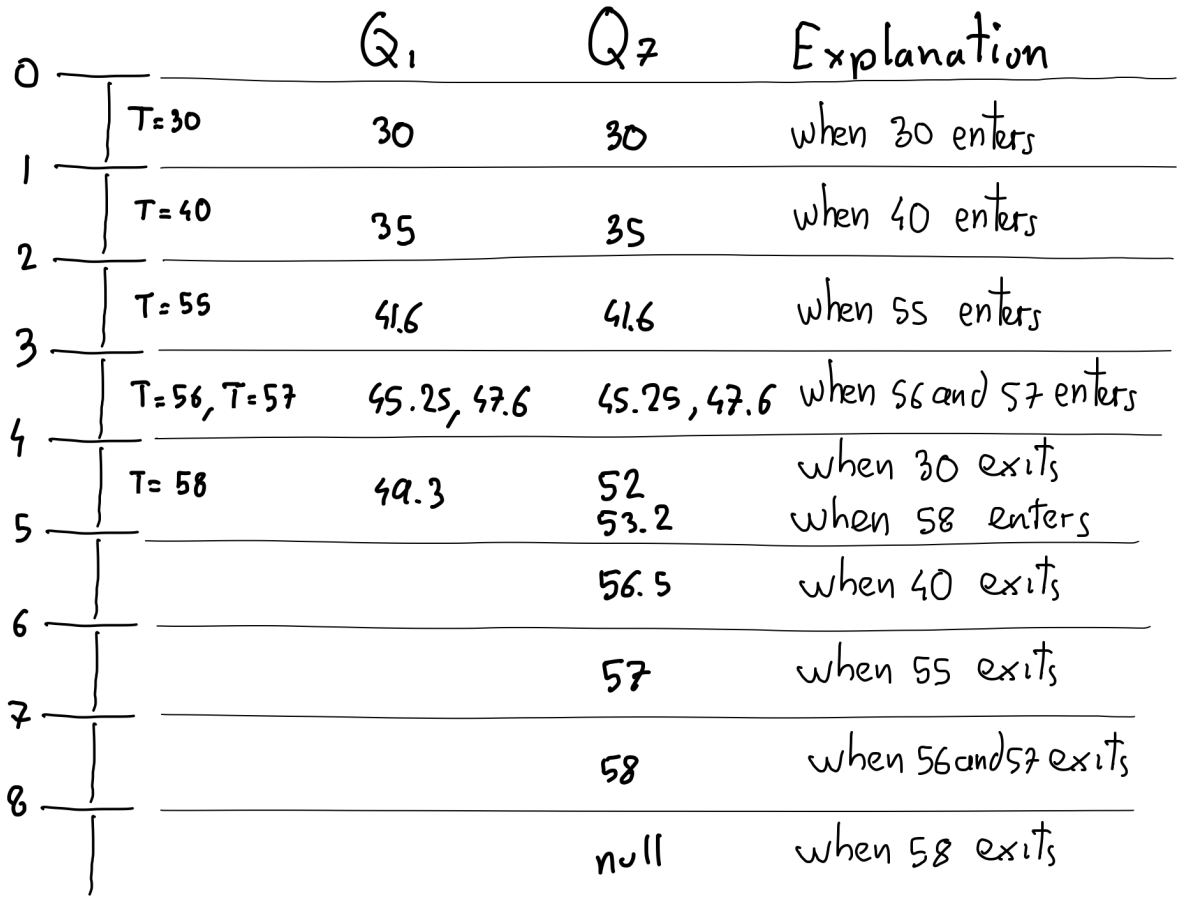
The average temperature observed by each sensor in the last 4 seconds (a.k.a. **logical sliding window**)

Assumption: the average should change as soon as they receive a new event

```
@Name('Q7')
select sensor, avg(temperature)
from TemperatureSensorEvent.win:time(4 seconds)
group by sensor;
```

Q1's vs. Q7's results

I am elaborating more on Q2 (landmark window) and the difference with Q7 (sliding window).



Q8

the moving average of the last 4 temperature events (a.k.a. **physical sliding window**)

```
@Name('Q8')
select sensor, avg(temperature)
from TemperatureSensorEvent.win:length(4)
group by sensor;
```

Q9

the average temperature of the last 4 seconds every 4 seconds (a.k.a. **logical tumbling window**)

```
@Name('Q9')
select sensor, avg(temperature)
from TemperatureSensorEvent.win:time_batch(4 seconds)
group by sensor;
```

Q10

the moving average of the last 4 temperature events every 4 events (a.k.a. **physical tumbling window**)

```
@Name('Q10')
select sensor, avg(temperature)
from TemperatureSensorEvent.win:length_batch(4)
group by sensor;
```

Q11

the average temperature of the last 4 seconds every 2 seconds (a.k.a. **logical hopping window**)

```
@Name('Q11')
select sensor, avg(temperature)
from TemperatureSensorEvent.win:time(4 seconds)
group by sensor
output snapshot every 2 seconds;
```

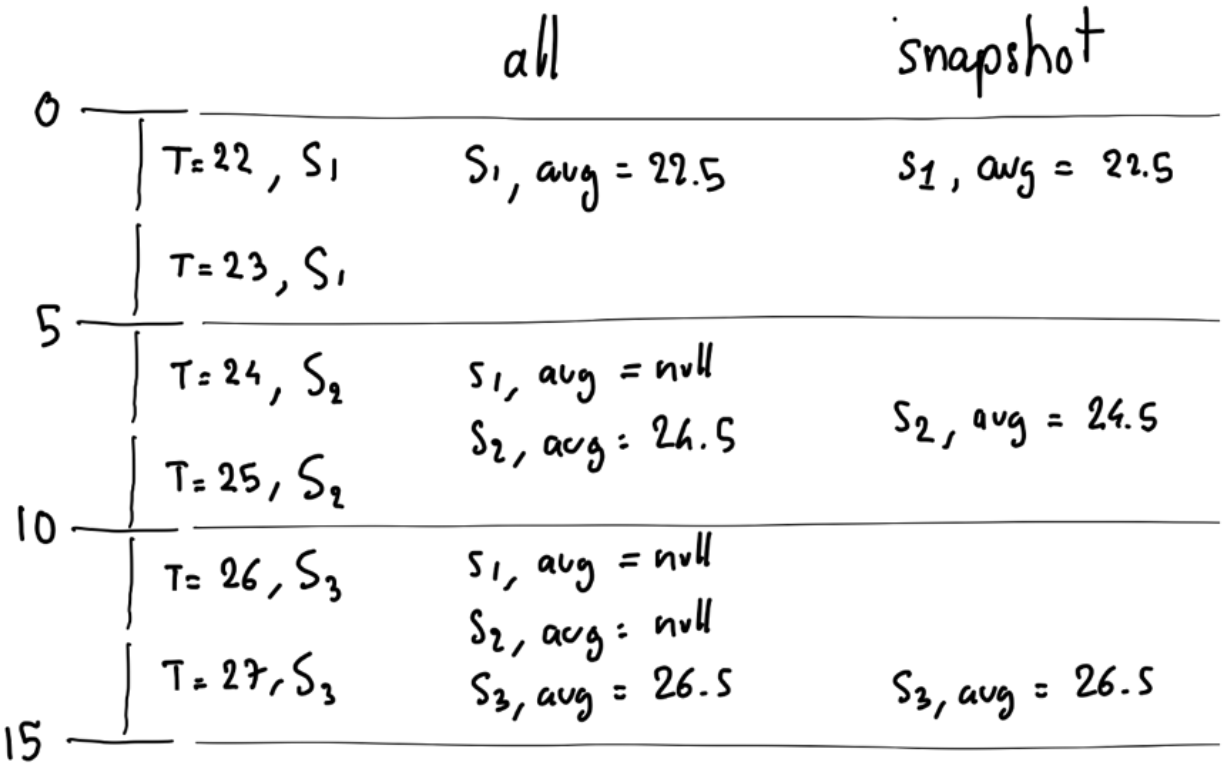
Deep dive into the reporting policy specified by the `output ... every` clause

first some data

```
TemperatureSensorEvent={sensor='S1', temperature=22}
TemperatureSensorEvent={sensor='S1', temperature=23}
t=t.plus(5 seconds)
TemperatureSensorEvent={sensor='S2', temperature=24}
TemperatureSensorEvent={sensor='S2', temperature=25}
t=t.plus(5 seconds)
TemperatureSensorEvent={sensor='S3', temperature=26}
TemperatureSensorEvent={sensor='S3', temperature=27}
t=t.plus(5 seconds)
```

Let's see the different behavior of the following queries.

```
@Name('Q.agg.all')
select sensor, avg(temperature) as avgTemp
from TemperatureSensorEvent.win:time(10 seconds)
group by sensor
output all every 5 seconds;

@Name('Q.agg.snapshot')
select sensor, avg(temperature) as avgTemp
from TemperatureSensorEvent.win:time(10 seconds)
group by sensor
output snapshot every 5 seconds;
```



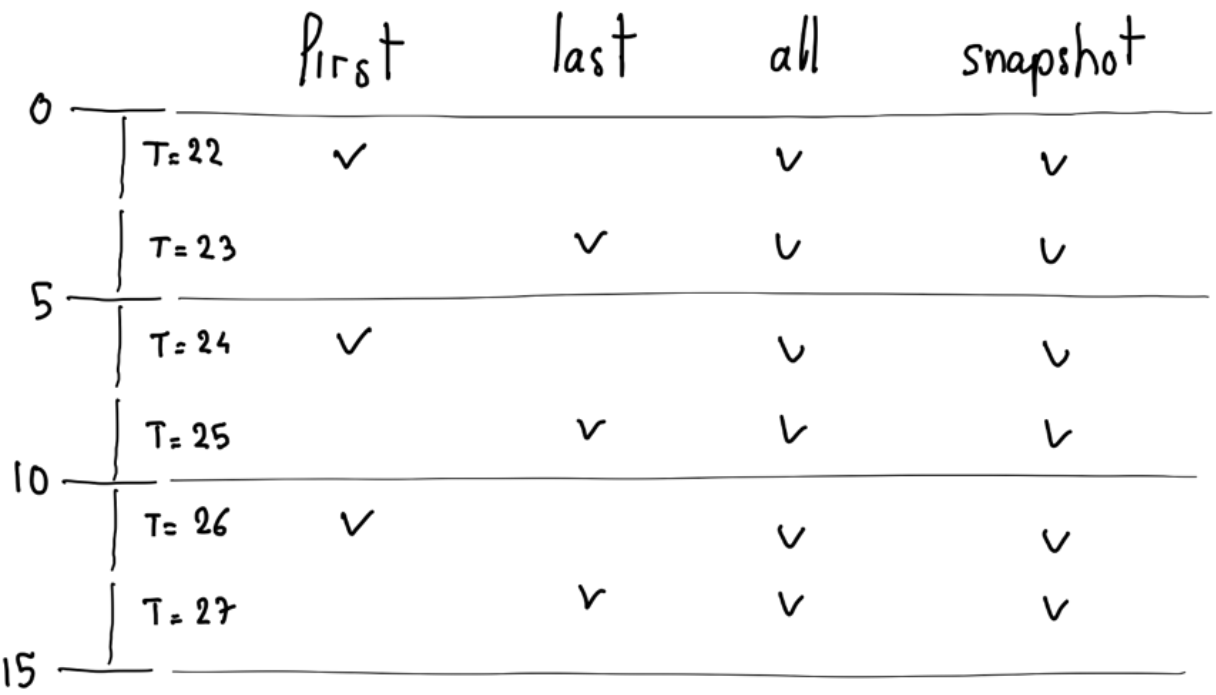
As you can see, `snapshot` and `all` *do not emit the same results*.

If you want to see a difference between `first` and `last`, we can examine the window's content.

```
@Name('Q.w.first')
select *
from TemperatureSensorEvent.win:time(10 seconds)
output first every 5 seconds;

@Name('Q.w.last')
select *
from TemperatureSensorEvent.win:time(10 seconds)
output last every 5 seconds;
```

Notably, in this case, `snapshot` and `all` *emit the same results* because they start from the same window. In the aggregation queries, they treat differently the internal state of the query.

```
@Name('Q.w.all')
select *
from TemperatureSensorEvent.win:time(10 seconds)
output all every 5 seconds;

@Name('Q.w.snapshot')
select *
from TemperatureSensorEvent.win:time(10 seconds)
output snapshot every 5 seconds;
```



Q12

Let's move on to the pattern matching part (a.k.a., the part of the language for Complex Event Processing)

Remember the running example: find every smoke event followed by a temperature above 50 °C within 2 minutes.

There is a particular operator to say *followed by* (syntactically `->`) to use within the `pattern` clause

Let's try using it naively.

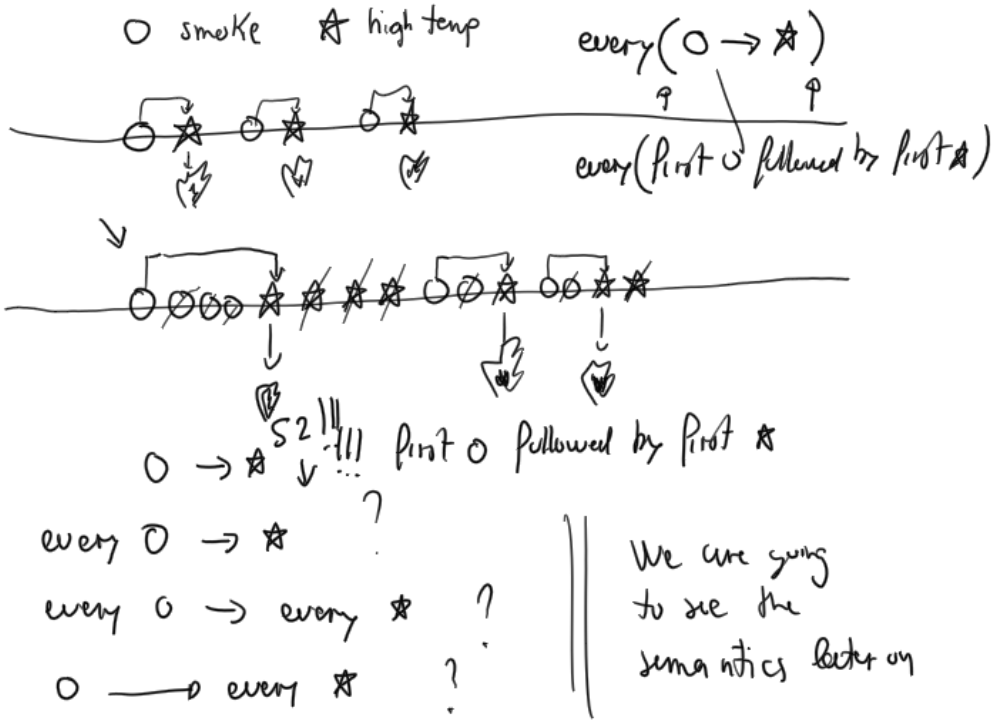
```
@Name('Q12.naive')
select *
from pattern [
  s = SmokeSensorEvent(smoke=true)
  ->
  TemperatureSensorEvent(temperature > 50, sensor=s.sensor)
]
;
```

Is this what we want?

Indeed this *tames the torrent effect*, but is there a way to have more results?

yes, using the `every` clause

```
@Name('Q12.every')
select *
from pattern [
 every (
  s = SmokeSensorEvent(smoke=true)
  ->
  TemperatureSensorEvent(temperature > 50, sensor=s.sensor)
)
]
;
```



Moreover, you may not like the payload of the events generated by this query.

You can create what you want, making a *projection* as in SQL.

```
@Name('Q12.every.projection')
select s.sensor AS sensor, t.temperature AS temperature, s.smoke as smok
from pattern [
 every (
  s = SmokeSensorEvent(smoke=true)
  ->
  t = TemperatureSensorEvent(temperature > 50, sensor=s.sensor)
)
]
;
```

Now, we need to insert the results in the FireEvent stream.

```
@Name('Q12.insert')
insert into FireEvent
select a.sensor AS sensor,
         a.smoke AS smoke,
         b.temperature AS temperature
from pattern [
   every (
               a = SmokeSensorEvent(smoke=true)
               ->
               b = TemperatureSensorEvent(temperature > 50,  sensor=a.s
               )
];
```

One last step and we have Q13 finalized. Let's add the "*within 2 minutes*" constraint.

```
insert into FireEvent
select s.sensor as sensor, s.smoke as smoke, t.temperature as temperatur
from pattern [
   every (
    s=SmokeSensorEvent(smoke=true)
      -> (
    t=TemperatureSensorEvent(temperature>50,sensor=s.sensor)
    where timer:within(2 seconds)
    )
  )
]
;
```

NOTE: alter some statements that advance the time to change the results. E.g., change one of the last three `t=t.plus(1 seconds)` in `t=t.plus(3 seconds)`

Q13

We are very close to the solution of the running example; we "just" need to count the number of events generated by the previous query over a sliding window of 10 seconds. So let's count the results of `Q13`

```
@Name('Q13')
select count(*)
from FireEvent.win:time(10 seconds);
```